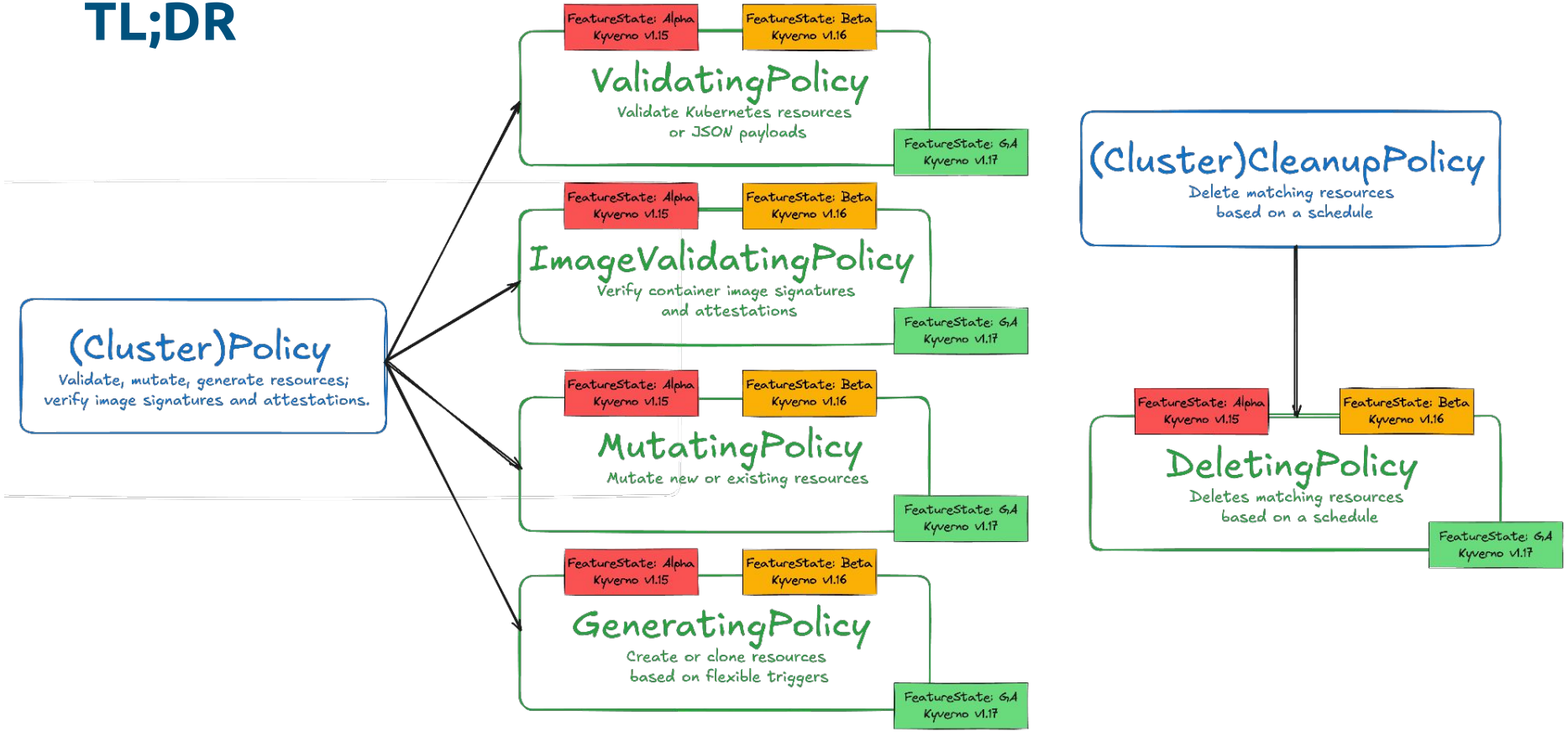
KubeCon CloudNativeCon Europe 2026

KyvernoCon EUROPE

Beyond Match & Pattern: Mastering Kyverno with CEL

TL;DR


(Cluster)Policy
Validate, mutate, generate resources; verify image signatures and attestations.
FeatureState: Alpha Kyverno v1.15
FeatureState: Beta Kyverno v1.16
ValidatingPolicy
Validate Kubernetes resources or JSON payloads
FeatureState: GA Kyverno v1.17
FeatureState: Alpha Kyverno v1.15
FeatureState: Beta Kyverno v1.16
ImageValidatingPolicy
Verify container image signatures and attestations
FeatureState: GA Kyverno v1.17
FeatureState: Alpha Kyverno v1.15
FeatureState: Beta Kyverno v1.16
MutatingPolicy
Mutate new or existing resources
FeatureState: GA Kyverno v1.17
FeatureState: Alpha Kyverno v1.15
FeatureState: Beta Kyverno v1.16
GeneratingPolicy
Create or clone resources based on flexible triggers
FeatureState: GA Kyverno v1.17
(Cluster)CleanupPolicy
Delete matching resources based on a schedule
FeatureState: Alpha Kyverno v1.15
FeatureState: Beta Kyverno v1.16
DeletingPolicy
Deletes matching resources based on a schedule
FeatureState: GA Kyverno v1.17

KUBERMATIC

WHOAMI?

Koray Oksay

- Kubernetes Consultant and Trainer
- CNCF & Platform Engineering Ambassador
- KCD and DevOpsDays Istanbul Org.
- Contributor at #sig-k8s-infra and apisnoop/verify-conformance
- Maintaining CNCF Hosted GitHub Runners

KUBERMATIC

Kubernetes 💙 CEL

So does Kyverno

What is CEL?

- Google's **`Common Expression Language`**
- https://cel.dev
- Fast, Portable, Extensible, and Safe
- Declarative expressions, evaluated at runtime
- Common Expression Language in Kubernetes
- Kubernetes support:
  - CRD Validation Rules (since k8s v1.25)
  - ValidatingAdmissionPolicy (since k8s v1.30)

Syntax - Expressions

```
"ghcr.io/kyverno/kyverno".startsWith("ghcr.io") // true

"nginx:latest".endsWith(":latest") // true
```

Syntax - Lists

```
[
  "ghcr.io/demo/foo",
  "ghcr.io/demo/bar",
  "docker.io/demo/foobar"
].all(img, img.startsWith("ghcr.io"))  // false
```

Syntax - Objects

```
[
  {
    "containerName": "foo",
    "containerImage": "ghcr.io/demo/foo"
  },
  {
    "containerName": "bar",
    "containerImage": "ghcr.io/demo/bar"
  }
].map(c, c.containerImage).all(img, img.startsWith("ghcr.io"))   // true
```

```
"ghcr.io/demo/foobar:latest".startsWith("ghcr.io") &&
  !"ghcr.io/demo/foobar:latest".endsWith(":latest")   // false
```

https://github.com/google/cel-spec/blob/master/doc/langdef.md

https://playcel.undistro.io/

CEL Expression — Blank

```
[
  {
    "containerName": "foo",
    "containerImage": "ghcr.io/demo/foo"
  },
  {
    "containerName": "bar",
    "containerImage": "ghcr.io/demo/bar"
  }
].map(c, c.containerImage)
```

Output — Cost: 29

```
["ghcr.io/demo/foo","ghcr.io/demo/bar"]
```

CEL Expression — Blank

```
[
  {
    "containerName": "foo",
    "containerImage": "ghcr.io/demo/foo"
  },
  {
    "containerName": "bar",
    "containerImage": "ghcr.io/demo/bar"
  }
].map(c, c.containerImage).all(img, img.startsWith("ghcr.io"))
```

Output — Cost: 42

```
true
```

CEL PLAYGROUND | CEL Expression | Modes | Share | Visit our GitHub

CEL Expression

Disallow HostPorts

```
// According the Pod Security Standards, HostPorts should be disallowed entirely.
// https://kubernetes.io/docs/concepts/security/pod-security-standards/#baseline

object.spec.template.spec.containers.all(container,
  !has(container.ports) ||
  container.ports.all(port,
    !has(port.hostPort) ||
    port.hostPort == 0
  )
)

```

Input

Run

```
object:
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: nginx
  spec:
    template:
      metadata:
        name: nginx
        labels:
          app: nginx
      spec:
        containers:
          - name: nginx
            image: nginx
            ports:
              - containerPort: 80
                hostPort: 80 # the expression looks for this field
    selector:
      matchLabels:
        app: nginx
```

Output

Cost: 22

```
false
```

What about Kyverno?

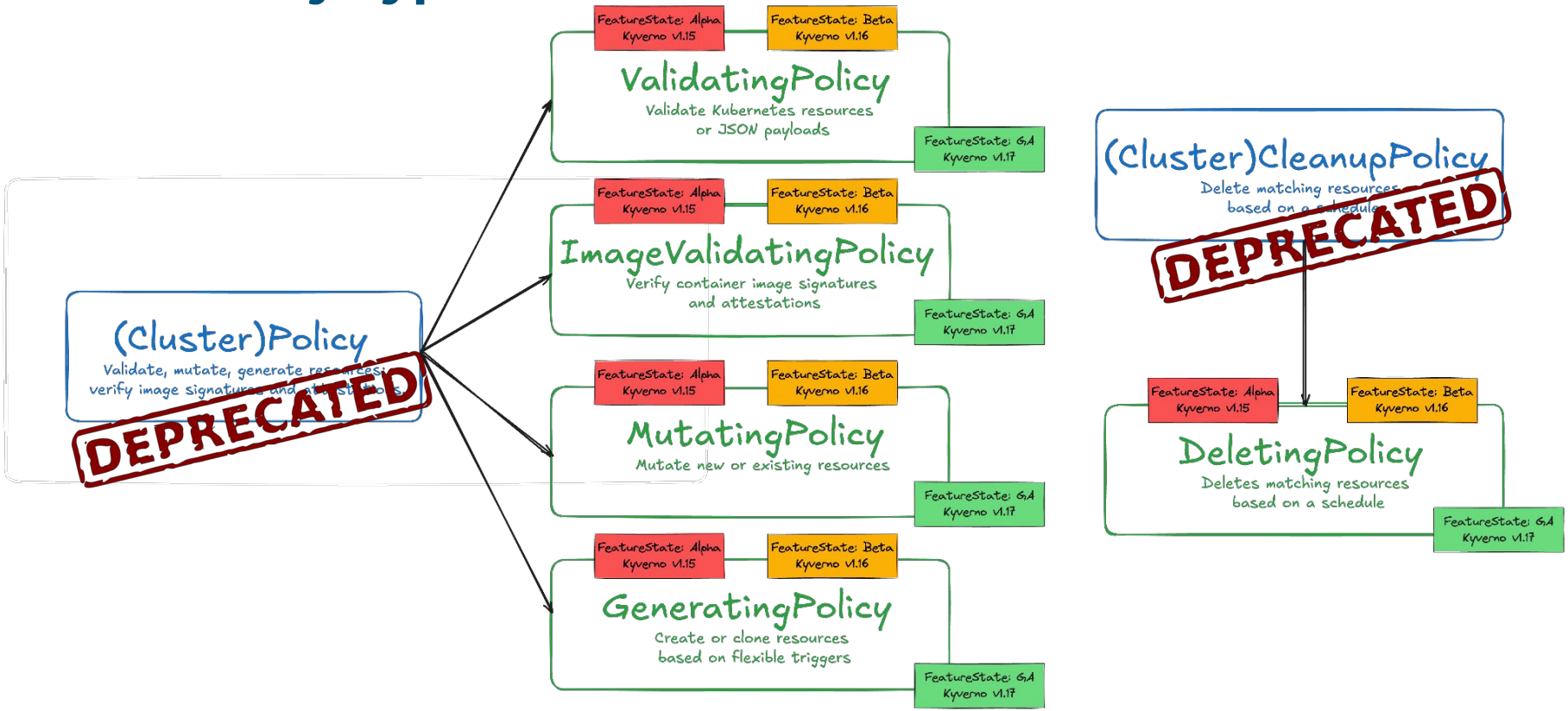
```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-latest-tag
spec:
  validationFailureAction: Audit
  background: true
  rules:
  - name: require-image-tag
    match:
      any:
      - resources:
          kinds:
          - Pod
    validate:
      message: "An image tag is required."
      foreach:
        - list: "request.object.spec.containers"
          pattern:
            image: "*:*"
  - name: validate-image-tag
    match:
      any:
      - resources:
          kinds:
          - Pod
    validate:
      message: "Using image tag 'latest' is not allowed."
      foreach:
        - list: "request.object.spec.containers"
          pattern:
            image: "!*:latest"
```

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-latest-tag
spec:
  validationFailureAction: Audit
  background: true
  rules:
  - name: require-and-validate-image-tag
    match:
      any:
      - resources:
          kinds:
          - Pod
          operations:
          - CREATE
          - UPDATE
    validate:
      cel:
        expressions:
          - expression: "object.spec.containers.all(c, c.image.contains(':'))"
            message: "An image tag is required."
          - expression: "object.spec.containers.all(c, !c.image.endsWith(':latest'))"
            message: "Using a mutable image tag e.g. 'latest' is not allowed."
```

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: check-deployment-replicas
spec:
  validationFailureAction: Enforce
  background: false
  rules:
    - name: deployment-replicas
      match:
        any:
        - resources:
            kinds:
              - Deployment
      validate:
        cel:
          paramKind:
            apiVersion: v1
            kind: ConfigMap
          paramRef:
            name: replica-limit
            parameterNotFoundAction: "Deny"
          expressions:
            - expression: "object.spec.replicas <= int(params.data.maxReplicas)"
              messageExpression:  "'Deployment spec.replicas must be less than ' + string(params.data.maxReplicas)"
```

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: replica-limit
data:
  maxReplicas: 3
```

New Policy Types



Deprecation Schedule for Legacy Types

The `ClusterPolicy` and `CleanupPolicy` will be supported for multiple releases:

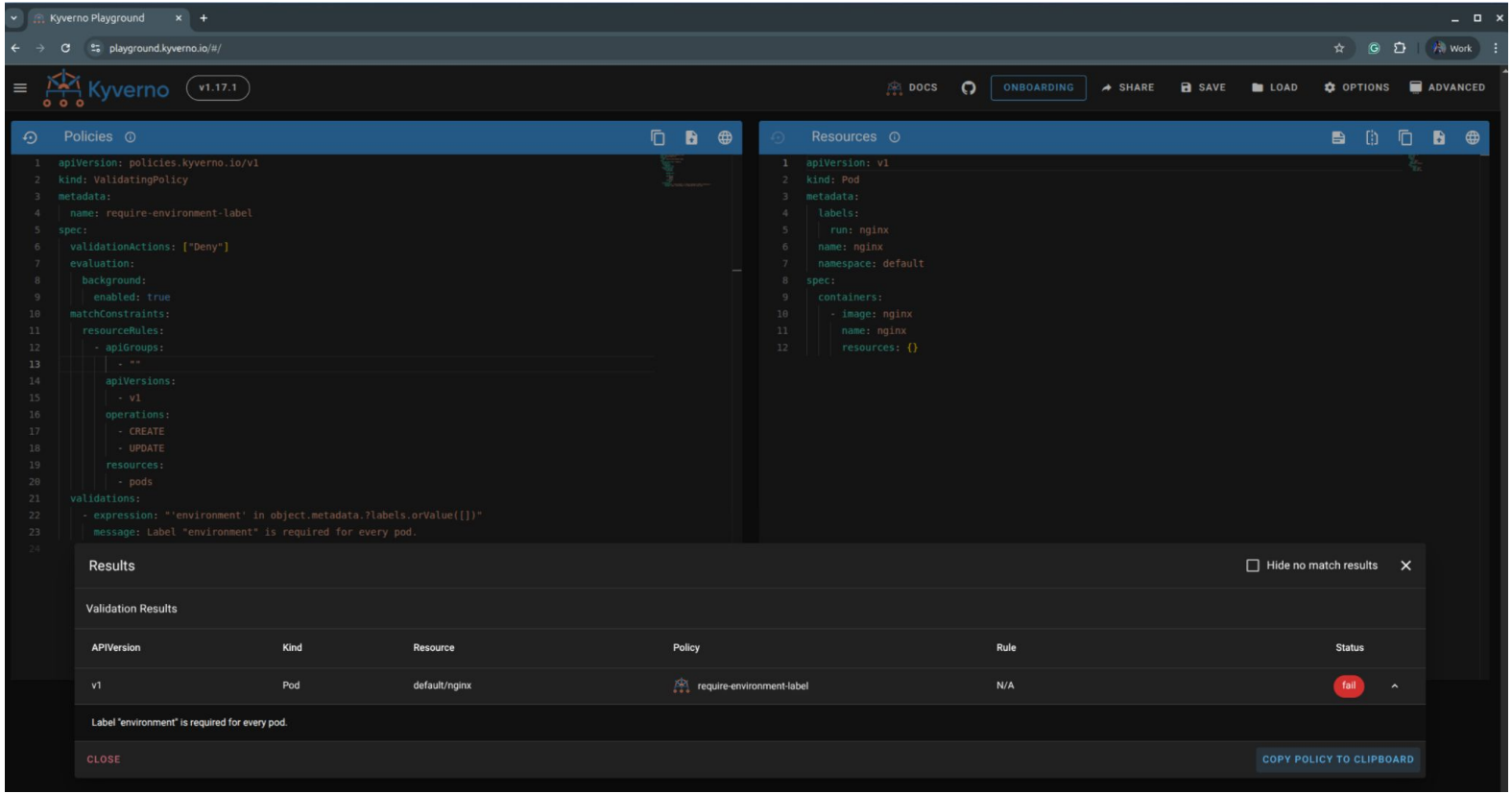
Release	Date (estimated)	Status
v1.17	Jan 2026	Marked for deprecation
v1.18	Apr 2026	Critical fixes only
v1.19	Jul 2026	Critical fixes only
v1.20	Oct 2026	Planned for removal

```
$> kubectl api-resources --api-group=kyverno.io
NAME                     SHORTNAMES   APIVERSION      NAMESPACED   KIND
cleanuppolicies          cleanpol     kyverno.io/v2   true         CleanupPolicy
clustercleanuppolicies   ccleanpol    kyverno.io/v2   false        ClusterCleanupPolicy
clusterpolicies          cpol         kyverno.io/v1   false        ClusterPolicy
globalcontextentries     gctxentry    kyverno.io/v2   false        GlobalContextEntry
policies                 pol          kyverno.io/v1   true         Policy
policyexceptions         polex        kyverno.io/v2   true         PolicyException
updaterequests           ur           kyverno.io/v2   true         UpdateRequest

$> kubectl api-resources --api-group=policies.kyverno.io
NAME                                SHORTNAMES   APIVERSION                NAMESPACED   KIND
deletingpolicies                    dpol         policies.kyverno.io/v1    false        DeletingPolicy
generatingpolicies                  gpol         policies.kyverno.io/v1    false        GeneratingPolicy
imagevalidatingpolicies             ivpol        policies.kyverno.io/v1    false        ImageValidatingPolicy
mutatingpolicies                    mpol         policies.kyverno.io/v1    false        MutatingPolicy
namespaceddeletingpolicies          ndpol        policies.kyverno.io/v1    true         NamespacedDeletingPolicy
namespacedgeneratingpolicies        ngpol        policies.kyverno.io/v1    true         NamespacedGeneratingPolicy
namespacedimagevalidatingpolicies   nivpol       policies.kyverno.io/v1    true         NamespacedImageValidatingPolicy
namespacedmutatingpolicies          nmpol        policies.kyverno.io/v1    true         NamespacedMutatingPolicy
namespacedvalidatingpolicies        nvpol        policies.kyverno.io/v1    true         NamespacedValidatingPolicy
policyexceptions                                 policies.kyverno.io/v1    true         PolicyException
validatingpolicies                  vpol         policies.kyverno.io/v1    false        ValidatingPolicy
```

KUBERMATIC

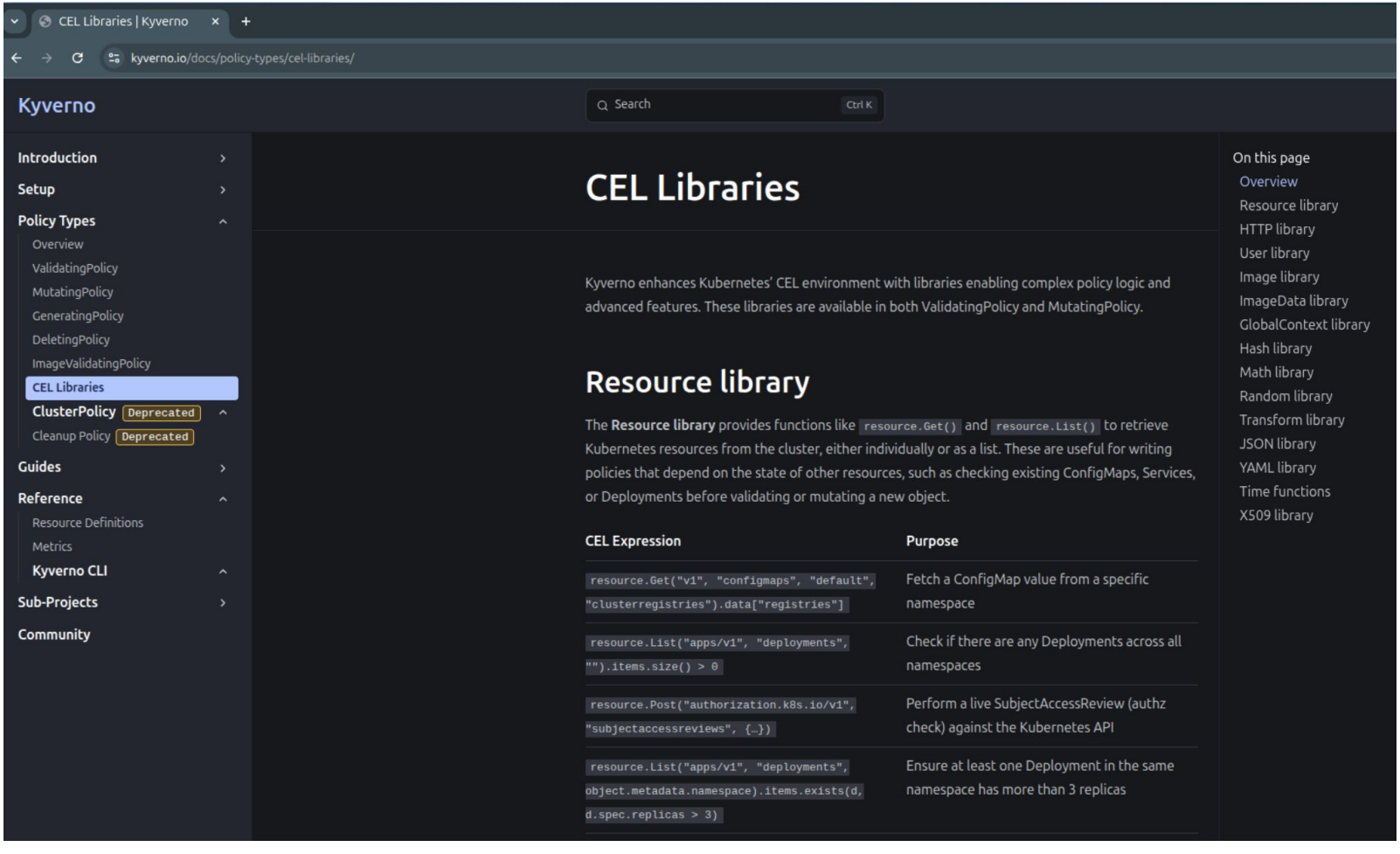
https://playground.kyverno.io



KUBERMATIC

Quick Demo

What actually we gain from CEL?


CEL Libraries | Kyverno
kyverno.io/docs/policy-types/cel-libraries/
Kyverno
Search
Ctrl K
Introduction
Setup
Policy Types
Overview
ValidatingPolicy
MutatingPolicy
GeneratingPolicy
DeletingPolicy
ImageValidatingPolicy
CEL Libraries
ClusterPolicy Deprecated
Cleanup Policy Deprecated
Guides
Reference
Resource Definitions
Metrics
Kyverno CLI
Sub-Projects
Community
CEL Libraries
Kyverno enhances Kubernetes' CEL environment with libraries enabling complex policy logic and advanced features. These libraries are available in both ValidatingPolicy and MutatingPolicy.
Resource library
The Resource library provides functions like resource.Get() and resource.List() to retrieve Kubernetes resources from the cluster, either individually or as a list. These are useful for writing policies that depend on the state of other resources, such as checking existing ConfigMaps, Services, or Deployments before validating or mutating a new object.
CEL Expression
Purpose
resource.Get("v1", "configmaps", "default", "clusterregistries").data["registries"]
Fetch a ConfigMap value from a specific namespace
resource.List("apps/v1", "deployments", "").items.size() > 0
Check if there are any Deployments across all namespaces
resource.Post("authorization.k8s.io/v1", "subjectaccessreviews", {…})
Perform a live SubjectAccessReview (authz check) against the Kubernetes API
resource.List("apps/v1", "deployments", object.metadata.namespace).items.exists(d, d.spec.replicas > 3)
Ensure at least one Deployment in the same namespace has more than 3 replicas
On this page
Overview
Resource library
HTTP library
User library
Image library
ImageData library
GlobalContext library
Hash library
Math library
Random library
Transform library
JSON library
YAML library
Time functions
X509 library

```yaml
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: unique-ingress-host
spec:
  validationActions:
    - Deny
  evaluation:
    background:
      enabled: false
  matchConstraints:
    resourceRules:
      - apiGroups: ["networking.k8s.io"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["ingresses"]
  variables:
    - name: knownIngresses
      expression: resource.List("networking.k8s.io/v1", "ingresses", "").items.orValue([])
    - name: knownHosts
      expression: "variables.knownIngresses.map(i, i.spec.rules.map(r, r.host))"
    - name: desiredHosts
      expression: "object.spec.rules.map(r, r.host)"
  validations:
    - expression: "!variables.knownHosts.exists_one(hosts, sets.intersects(hosts, variables.desiredHosts))"
      message: "Cannot reuse a host across multiple ingresses"
```

```yaml
apiVersion: v1
items:
  - apiVersion: networking.k8s.io/v1
    kind: Ingress
    metadata:
      name: ingress-1
      namespace: default
    spec:
      rules:
        - host: foo.com
          http:
            paths:
              - backend:
                  service:
                    name: service1
                    port:
                      number: 80
                path: /foo
                pathType: Prefix
    status:
      loadBalancer: {}
kind: List
metadata:
  resourceVersion: ""
```

This example uses CEL's `map()` and `filter()` functions to conditionally add environment variables only to containers that use nginx images.

```yaml
apiVersion: policies.kyverno.io/v1
kind: MutatingPolicy
metadata:
  name: foreach-conditional
spec:
matchConstraints:
  resourceRules:
      - apiGroups: [""]
      apiVersions: ["v1"]
        resources: ["pods"]
      operations: ["CREATE", "UPDATE"]
  mutations:
    - patchType: JSONPatch
      jsonPatch:
        expression: |
          object.spec.containers.map(c,
            c.image.startsWith("nginx:") ?
            JSONPatch{
              op: "add",
              path: "/spec/containers/" + string(object.spec.containers.indexOf(c)) + "/env",
              value: [{"name": "NGINX_ENV", "value": "production"}]
            } : null
          ).filter(p, p != null)
```

```yaml
apiVersion: policies.kyverno.io/v1
kind: MutatingPolicy
metadata:
  name: foreach
spec:
  matchConstraints:
    resourceRules:
    - apiGroups: [ "" ]
      apiVersions: [ "v1" ]
      operations: [ "CREATE" ]
      resources: [ "pods" ]
  mutations:
  - patchType: ApplyConfiguration
          applyConfiguration:
      expression: >
        Object{
          spec: Object.spec{
            containers: object.spec.containers.map(container, Object.spec.containers{
              name: container.name,
              securityContext: Object.spec.containers.securityContext{
                allowPrivilegeEscalation: false
              }
            }
          }
        }
```

This example uses CEL's `map()` function to iterate through all containers in a pod and add a `securityContext` with `allowPrivilegeEscalation: false` to each container.

Conclusion

- Pattern matching gets you started
- CEL takes you the rest of the way
- Context-aware, DRY, and still readable
- No external tools. No sidecars. Just policy.
- Your platform deserves policies that think

Kyverno + CEL: policy-as-code that actually scales

THANK YOU!


koray@kubermatic.com


@koksay


linkedin.com/in/korayoksay